

JCONDO MOBILE

Roteiro da apresentação final

Webconferência 3 · 24/09/2026 · duração prevista: 12 a 13 minutos

O texto entre colchetes e em itálico é instrução para você: o que abrir, onde clicar. O texto recuado é a fala. Não precisa decorar palavra por palavra, mas ensaiar em voz alta duas ou três vezes com esse texto ajuda a segurar o tempo e a não travar.

1 ANTES DE ENTRAR NA CHAMADA

Faça esta lista uns quinze minutos antes. É o que evita perder tempo de apresentação procurando janela.

- 1) Suba a API: Prompt na pasta backend e o comando `mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=h2` . Espere aparecer "Started JcondoApplication" e minimize a janela, sem fechar.
- 2) Abra o emulador com o aplicativo já instalado, na tela de login, e deixe a janela pronta em segundo plano.
- 3) Faça o login uma vez com a Ana e saia, só para confirmar que está tudo respondendo.
- 4) Abra o navegador em três abas: o Swagger (`localhost:8080/swagger-ui/index.html`), o console do banco (`localhost:8080/h2-console`) e o repositório no GitHub.
- 5) Abra o Android Studio no arquivo `ReservaService.java`, com a consulta de conflito visível na tela.
- 6) Abra este roteiro em outra janela ou imprima.
- 7) Teste o compartilhamento de tela antes de começar.

Uma reserva de segurança

Antes de apresentar, entre com a Ana e reserve um horário qualquer para uma data próxima. Isso garante que, na hora de mostrar a grade de horários, exista pelo menos um horário marcado como Ocupado. Uma grade toda livre não demonstra nada.

2 ROTEIRO

0:00 – 1:30 · Abertura e o problema

[Tela compartilhada: o aplicativo no emulador, tela de login]

Boa noite. Meu nome é Jorge, e vou apresentar o JCondo Mobile, que é o aplicativo que desenvolvi a partir do projeto da primeira atividade.

A ideia veio de uma situação que eu vejo no condomínio onde moro. A comunicação entre morador e administração acontece por caminhos improvisados. Aviso de manutenção sai em papel colado no elevador. Reserva do salão de festas é anotada num caderno na portaria. Reclamação de lâmpada queimada vira mensagem em grupo, se perde no meio das conversas e ninguém sabe se foi resolvida.

Isso gera três problemas que se repetem. O morador não fica sabendo de um aviso importante. Duas famílias reservam o salão para o mesmo sábado. E a ocorrência que o morador relatou não tem como ser cobrada depois, porque não existe protocolo nem status.

O JCondo junta essas três coisas em um lugar só: comunicados, reservas de área comum e registro de ocorrências. Vou mostrar funcionando primeiro, e depois entrar na parte técnica.

1:30 – 5:30 · Demonstração

[Faça o login: ana.souza@email.com / 123456]

Entro com o e-mail e a senha de um morador. O login devolve um token de sessão que fica guardado no aparelho, então da próxima vez o aplicativo abre direto na tela inicial. Foi uma decisão de experiência: quem abre o app para ver um aviso não deveria digitar a senha toda vez.

[Tela inicial]

A tela inicial responde de imediato às três perguntas mais comuns: tem aviso novo, tenho reserva marcada, tenho ocorrência em aberto. Abaixo vem o próximo compromisso e os avisos recentes.

Uma coisa que vale destacar aqui: essa tela faz uma chamada só à API. Eu tinha feito com quatro chamadas, uma para cada bloco, e a tela montava aos pedaços. Criei um endpoint que devolve tudo junto, o `/api/home/resumo`. Menos tráfego, menos bateria e a tela carrega de uma vez.

[Toque em um aviso, mostre o texto completo, volte]

Os avisos vêm da administração, ordenados do mais recente para o mais antigo. Repare na etiqueta de prioridade: ela tem cor, mas tem o texto junto. Isso é proposital. Se a informação estivesse só na cor, quem tem dificuldade para distinguir cores perderia o aviso urgente.

[Aba Reservas → botão NOVA RESERVA]

Agora a parte mais interessante. A reserva de área comum tem três passos: escolho a área, escolho a data, escolho o horário.

[Escolha uma área, toque em Escolher data, selecione um dia]

Quando escolho a data, o aplicativo consulta a agenda daquele dia no servidor e monta a grade de horários. Os horários ocupados aparecem, mas desabilitados e com a palavra "Ocupado". Prefiro mostrar em vez de esconder, porque sumir com opções dá a sensação de que o aplicativo perdeu informação.

[Selecione um horário livre e confirme]

Confirmando, o servidor grava e devolve a confirmação com data e horário. Essa confirmação explícita é o que fecha a ação para o usuário.

[Aba Ocorrências → NOVA OCORRÊNCIA]

As ocorrências funcionam de forma parecida. Escolho a categoria, descrevo o problema, e o sistema devolve um número de protocolo. Esse protocolo é o que resolve o terceiro problema que citei no começo: agora o morador tem como cobrar.

[Volte e abra a ocorrência da lâmpada queimada]

Aqui está uma ocorrência com status "Em andamento" e a resposta da administração. O morador acompanha sem precisar ligar para a portaria.

5:30 – 8:00 · Arquitetura

[Compartilhe a Figura 1 da documentação, ou desenhe as três camadas]

Vamos para a parte técnica. O sistema tem três camadas: o aplicativo Android, uma API REST em Spring Boot e o banco de dados.

O backend não foi feito do zero. Eu aproveitei o JCondo que já tinha desenvolvido no projeto integrador, que era um CRUD de moradores com Thymeleaf e uma API REST documentada com Swagger. Estendi ele: acrescentei quatro entidades, a autenticação por token e dezesseis endpoints. As telas web originais continuam funcionando.

[Abra o Swagger no navegador]

Esta é a API documentada. São dezesseis endpoints agrupados por assunto. Todos eles, com exceção do login, exigem o token no header Authorization.

Tem uma decisão de segurança que percorre todos os endpoints do morador: nenhum recebe o identificador do usuário por parâmetro. A rota é "barra api barra reservas barra minhas", e não "reservas interrogação morador id igual três". Quem é o dono da requisição sai sempre do token. Isso impede que alguém veja os dados de outro morador só trocando um número na URL.

A parte que amarra tudo do lado do aplicativo é um interceptor no cliente HTTP. Ele anexa o token em toda requisição que não seja o login. Nenhuma das telas precisa saber que o token existe, e não tem como esquecer de mandar em alguma chamada. Se um dia eu trocar para JWT assinado, mexo em um arquivo só.

8:00 – 10:30 · A regra de conflito de reserva

[Abra o ReservaService.java no Android Studio, na consulta de conflito]

Este é o ponto que eu considero o mais importante do trabalho, e quero gastar dois minutos nele.

Entre todos os requisitos, o mais delicado é impedir que duas reservas ocupem o mesmo horário na mesma área. É o único ponto do sistema em que dois usuários podem, de boa-fé, tentar fazer a mesma coisa ao mesmo tempo.

Repare que o aplicativo já mostra os horários ocupados em cinza. Seria tentador parar por aí e dizer que o problema está resolvido. Mas não está. Imagina dois moradores que abrem essa tela ao mesmo tempo. Os dois consultam a agenda, os dois veem o mesmo horário livre, e os dois confirmam. Se a verificação estivesse só no celular, os dois teriam gravado.

Por isso a decisão acontece no servidor, no momento da gravação. Esta é a consulta que roda no banco.

[Aponte para a consulta na tela]

Ela conta as reservas confirmadas daquela área, naquela data, cuja hora de início é menor que a hora de fim que eu pedi, e cuja hora de fim é maior que a hora de início que eu pedi. É a comparação cruzada de sobreposição de intervalos. Se o resultado for maior que zero, a API devolve HTTP 409 e a reserva não é gravada.

Tem um detalhe aqui que me custou tempo. A primeira versão que escrevi usava comparação com "menor ou igual" e "maior ou igual". Com isso, uma reserva das quatorze às dezesseis bloqueava a das dezesseis às dezoito, que na prática não conflita, porque uma termina exatamente quando a outra começa. Eu descobri isso escrevendo o teste automatizado, não usando o aplicativo.

[Abra o ReservaServiceTest.java]

São nove testes só para essa regra. O que apontou o erro foi este aqui, o "agenda deve marcar como indisponível apenas a faixa ocupada". Hoje ele é o que impede o comportamento de se perder de novo se alguém mexer no código depois.

[Se quiser rodar ao vivo: clique na seta verde ao lado da classe de teste]

E o aplicativo trata esse 409 de forma específica. Ele não mostra "erro 409". Ele mostra a frase que o servidor devolveu, que diz qual horário está ocupado em qual área, limpa a seleção e recarrega a agenda, deixando a tela coerente com o servidor. O morador não precisa sair e voltar para entender o que houve.

10:30 – 12:00 · Experiência do usuário e testes

[Volte para o aplicativo. Desligue o Wi-Fi do emulador, ou pare a API]

Sobre experiência do usuário, quero mostrar uma coisa que não estava prevista no projeto e apareceu durante o desenvolvimento.

Toda lista que vem de API tem quatro estados: carregando, com conteúdo, vazia e com erro. Se você trata só o segundo, o usuário encontra uma tela branca e não sabe se o app travou, se a internet caiu ou se realmente não tem nada.

[Puxe a tela para atualizar, com a API parada]

Aqui a API está parada. Em vez de tela branca ou de uma exceção em inglês, o morador lê que não foi possível falar com o servidor, e tem um botão para tentar de novo. Fiz um componente único para esses três estados, reutilizado nas quatro telas de lista, então todas se comportam igual.

[Religue a API e mostre a tela voltando ao normal]

Outros cuidados de acessibilidade: os alvos de toque têm no mínimo quarenta e oito dp, o contraste entre texto e fundo fica acima de quatro e meio para um, e nenhuma informação depende só de cor.

No total são vinte testes automatizados com JUnit e Mockito, cobrindo a regra de reserva, o hash de senha e o ciclo de vida do token. Rodam sem banco, porque o repositório é simulado.

12:00 – 13:00 · Limitações, evolução e fechamento

Antes de encerrar, quero registrar o que ficou de fora de propósito, porque acho mais honesto do que deixar implícito.

As senhas usam SHA-256 com sal. Isso impede guardar senha em texto puro, mas um algoritmo rápido é vulnerável a força bruta em placa de vídeo. Em produção o correto seria BCrypt ou Argon2. As sessões ficam em memória, então reiniciar o servidor derruba os logins. E as notificações são locais: o push de verdade, que chega com o app fechado, exigiria Firebase Cloud Messaging.

Sobre a continuidade, três decisões deixam o projeto preparado. A lógica de negócio está no servidor, então uma versão iOS consumiria a mesma API sem duplicar regra. A separação em camadas faz com que uma funcionalidade nova entre sempre nos mesmos lugares. E os componentes compartilhados, como o tratador de erros e o de estados de tela, fazem qualquer tela nova já nascer com esses comportamentos prontos.

Fechando: o aplicativo entrega o MVP inteiro que eu tinha definido na primeira atividade. O que eu levo deste trabalho não é ter aprendido a chamar um endpoint. É ter entendido onde colocar cada responsabilidade: o servidor decide, o aplicativo apresenta e antecipa. É essa divisão que mantém a regra do condomínio válida mesmo quando dois moradores agem ao mesmo tempo.

O código está no GitHub, com a documentação e o guia de execução. Obrigado, e fico à disposição para perguntas.

3 PERGUNTAS PROVÁVEIS

Respostas curtas para as perguntas que costumam vir. Responda em uma ou duas frases e pare; se quiserem mais, perguntem.

Tabela 1 - Perguntas e respostas preparadas

Pergunta	Resposta
Por que Android e não híbrido?	Porque o público-alvo é o condomínio onde moro, e a maioria usa Android, incluindo aparelhos de entrada. E porque a lógica está no servidor: uma versão iOS futura consome a mesma API, então a escolha não vira amarra.
Por que Java e não Kotlin?	Java é o que foi trabalhado na disciplina. Kotlin seria uma escolha natural em um projeto novo hoje, mas aqui eu preferi aplicar o conteúdo estudado em vez de dividir esforço com uma linguagem nova.
O app funciona sem internet?	Não. Ele guarda o token e as preferências no aparelho, mas os dados vêm todos da API. O que ele faz bem é avisar com clareza quando está sem conexão, em vez de mostrar tela branca. Cache local está na lista de evolução.
Como você garantiu que um morador não vê os dados de outro?	Nenhum endpoint do morador recebe identificador por parâmetro. O interceptor valida o token, descobre quem é o dono da sessão e é esse dono que o controller usa. Não tem número na URL para trocar.
E se dois usuários reservarem no mesmo instante?	Um grava e o outro recebe 409. A verificação de sobreposição roda no banco, no momento da gravação, e não no aplicativo. Tem teste automatizado cobrindo esse caso.
Por que SHA-256 e não BCrypt?	Para não trazer o Spring Security só por causa do hash, o que aumentaria bastante o escopo. Está registrado como limitação na documentação, e é a primeira coisa que eu mudaria antes de colocar em produção.
Quantos endpoints tem a API?	Dezesseis, mais o CRUD de moradores herdado da etapa anterior. Todos documentados no Swagger e no arquivo API.md do repositório.
Você testou em celular físico?	Sim. O endereço da API é configurável na tela de login, então o mesmo APK roda no emulador e em um aparelho na mesma rede Wi-Fi, sem recompilar.
Quanto tempo levou?	Cerca de três semanas entre o projeto e a implementação. A parte que mais consumiu tempo não foi a interface, foi acertar a regra de conflito e o tratamento de erro.
Por que o banco é o H2 e não o MySQL?	O projeto roda nos dois: é a mesma aplicação, muda só o arquivo de configuração. Estou demonstrando com H2 porque ele recarrega os dados de exemplo a cada inicialização, o que deixa a demonstração previsível. O script de criação do banco MySQL está no repositório, em backend/database.
O H2 é um banco de verdade?	É um banco relacional completo, com tabelas, chaves estrangeiras e SQL. A diferença é que ele roda dentro do processo da aplicação e guarda os dados em memória, em vez de ser um servidor separado. Posso mostrar as tabelas no console, se quiser.

Fonte: o autor (2026).

4 SE DER PROBLEMA AO VIVO

Tabela 2 - Plano B para cada falha possível

Se acontecer	O que fazer
O emulador travar ou ficar lento demais.	Não insista. Diga "vou mostrar pelas telas do documento" e passe para os mockups da documentação. Continue a fala normalmente.
A API cair no meio da demonstração.	Aproveite: é exatamente a situação da seção de tratamento de erro. Mostre a mensagem que o app exibe e explique que foi assim que você tratou.
A internet oscilar e o compartilhamento travar.	Peça um minuto, feche e reabra o compartilhamento. Se persistir, siga narrando pelo documento aberto.
Esquecer o que ia falar.	Volte para a estrutura: problema, o que o app faz, como está montado, a regra principal, o que ficou de fora. Nessa ordem sempre dá para retomar.
O tempo estourar.	Corte o bloco de UX e testes (10:30 às 12:00) e vá direto para o fechamento. A regra de conflito é o que não pode faltar.
Sobrar tempo.	Mostre o teste automatizado rodando, ou abra o GitHub e passeie pela estrutura de pastas explicando a organização.

Fonte: o autor (2026).

5 CRONÔMETRO

Tabela 3 - Distribuição do tempo

Bloco	Duração	Acumulado
Abertura e problema	1min30	1:30
Demonstração do aplicativo	4min00	5:30
Arquitetura e API	2min30	8:00
Regra de conflito de reserva	2min30	10:30
UX e testes	1min30	12:00
Limitações e fechamento	1min00	13:00

Fonte: o autor (2026).

A janela é de dez a quinze minutos. Treze deixa folga para perguntas e para os imprevistos de qualquer apresentação ao vivo. Se você perceber que está atrasado no meio, o bloco de UX e testes é o que pode encolher sem prejudicar o conjunto.